Containers

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Exercises

Practical Lab: Working with Docker Compose

Objective: This lab will guide you through the fundamentals of creating, managing, and deploying applications using Docker Compose. You will apply the concepts of images, containers, volumes, and networking to build and run single and multi-service applications.

Prerequisites:

- A computer with a modern web browser and a text editor.
- Docker and Docker Compose installed (see installation section below).

Installing Docker on Debian

If you are using a Linux host, follow these steps in your terminal to install the latest version of Docker (you can install it manually, following these instructions).

To simplify the installation process, you can run the following bash script (works on Ubuntu and Debian). It will set up the Docker repository, install Docker Engine, and configure your user permissions:

`./classes/class_03/02_support/docker_setup.sh`

Tip: If you encounter a "permission denied" error when running `docker` commands, make sure you installed docker correctly and logged out/in again. As a quick workaround you can prefix commands with sudo, but configuring the group is the **recommended** approach.

Exercise 1: "Hello, World" with Docker Compose

Goal: Understand the basic structure of a `compose.yml` file and run a pre-built image.

1. From your working directory, create a new folder for this exercise:

   ```
   mkdir ex01
   cd ex01
   ```

2. Inside the folder, create a new file named `compose.yml` with the following content:

   ```
   services:
     hello:
       image: hello-world
   ```

3. Run the application:

   ```
   docker compose up
   ```

4. Observe the output. The `hello-world` container will start, print its message, and then exit.

5. Clean up the created container:

```
docker compose down
```

Verification: You should see the Docker welcome message that starts with "Hello from Docker!" in the terminal output after step 3. After step 5, running `docker compose ps` should show no containers.

Tip: The `compose.yml` file (previously called `docker-compose.yml`) is the standard filename that Docker Compose looks for. You can use a different filename with the `-f` flag: `docker compose -f myfile.yml up`.

Exercise 2: Building a Custom Web Server Image

Goal: Use a `Dockerfile` with Docker Compose to create a self-contained application image.

1. From your working directory, create the folder structure:

   ```
   mkdir -p ex02/my-website
   cd ex02
   ```

2. Inside `my-website`, create a file named `index.html`:

   ```
   <!DOCTYPE html>
   <html>
   <body>
       <h1>This page was built into the Docker image!</h1>
   </body>
   </html>
   ```

3. In the root of the `ex02` folder, create a `Dockerfile`:

   ```
   FROM nginx:alpine
   COPY ./my-website /usr/share/nginx/html
   ```

4. In the same folder, create your `compose.yml` file:

   ```
   services:
     webserver:
       build: .
       ports:
         - "8080:80"
   ```

5. Build and start the service. The `-d` flag runs it in the background (detached mode):

   ```
   docker compose up --build -d
   ```

6. Open your browser to `http://localhost:8080`. You should see your custom webpage.

Verification:

- Run `docker compose ps` to confirm the container is running and healthy.
- Run `docker compose images` to see the image that was built.
- Use `curl http://localhost:8080` as an alternative to the browser.

Tip: The `--build` flag forces Compose to rebuild the image before starting. Without it, Compose reuses the previously built image. Always use `--build` after changing the `Dockerfile` or any files that are copied into the image.

Cleanup: When you are done, stop and remove everything with:

```
docker compose down
```

Exercise 3: Live Development with Volumes

Goal: Understand how bind-mount volumes allow you to change your website's content without rebuilding the image.

1. From your working directory, create the folder structure:

```
mkdir -p ex03/my-website
cd ex03
```

2. Create a `my-website/index.html` file:

```
<!DOCTYPE html>
<html>
<body>
    <h1>Hello from a volume mount!</h1>
</body>
</html>
```

3. Create a `compose.yml` file. This time, we use the standard `nginx:alpine` image and mount our local folder as a volume. **No `Dockerfile` is needed.**

```
services:
  webserver:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./my-website:/usr/share/nginx/html:ro
```

4. Start the service:

```
docker compose up -d
```

5. Open your browser to `http://localhost:8080` to confirm it is working.
6. **Live Update:** While the container is running, **edit the `index.html` file** on your host machine. Change the heading to `<h1>Live update with a Volume!</h1>`.
7. Save the file and **refresh your browser**. The change appears instantly.

Verification: After editing `index.html`, you can verify the change from the command line:

```
curl http://localhost:8080
```

Tip: Notice the `:ro` (read-only) flag at the end of the volume mount. This is a best practice when the container should only read files from the host and never write to them. It prevents the container from accidentally modifying your source files.

Key Concept – Build vs. Volume: Compare Exercise 2 (build) with Exercise 3 (volume). Building creates a portable, self-contained image ideal for **production**. Volumes create a live link ideal for **development**. Understanding when to use each approach is fundamental.

Cleanup:

```
docker compose down
```

Exercise 4: Caching Rich Content with Varnish and NGINX

Goal: Build a two-tier web application with a Varnish HTTP cache serving a rich webpage from an NGINX backend. This exercise introduces multi-service compose files, service dependencies, and internal networking.

1. From your working directory, create the File Structure:

```
mkdir -p ex04/my-dynamic-website
cd ex04
```

2. Create the Web Content:
   - Find a fun animated GIF online and save it inside `my-dynamic-website` as `animation.gif`. For example, you can use this one: docker.gif.
   - Inside `my-dynamic-website`, create an `index.html` file to display the GIF:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Varnish Cache Test</title>
    <style> body { font-family: sans-serif; text-align: center; } </style>
</head>
<body>
    <h1>This page is being cached by Varnish!</h1>
    <img src="docker.gif" alt="Cached animation">
</body>
</html>
```

3. Create the Varnish Configuration:
    - Inside the `varnish` folder, create a file named `default.vcl`. This tells Varnish where to find the NGINX backend server:

      ```
      vcl 4.1;
      backend default {
          .host = "nginx";
          .port = "80";
      }
      ```

      - **Tip:** The `.host = "nginx"` line uses the service name from the compose file. Docker Compose automatically creates a DNS entry so that services can reach each other by name.
4. Create the Compose File:
    - In the root of your `ex04` folder, create the `compose.yml`:

      ```
      services:
        cache:
          image: varnish:stable
          volumes:
            - ./varnish:/etc/varnish:ro
          ports:
            - "8080:80"
          depends_on:
            - nginx
          restart: unless-stopped

        nginx:
          image: nginx:alpine
          volumes:
            - ./my-dynamic-website:/usr/share/nginx/html:ro
          # No ports exposed: nginx is only accessible
          # from within the Docker network
      ```
5. Run and Verify:
    - Start the services:

      `docker compose up -d`
    - Check that both services are running:

      `docker compose ps`

      You should see two containers listed, both in a “running” state.
    - Open your browser to `http://localhost:8080`. You should see your webpage with the GIF. The key here is that **Varnish** served you the page, not NGINX directly.
    - **See the cache in action:** Check the NGINX logs for the first request:

      `docker compose logs nginx`
    - Now, refresh your browser page several times. Check the `nginx` logs again:

```
docker compose logs nginx
```

You should see **no new log entries**, because Varnish is serving the content from its cache without contacting the NGINX backend.

- **Bonus – inspect the HTTP headers** to confirm caching:

```
curl -I http://localhost:8080
```

Look for headers like `X-Varnish` and Age. An Age value greater than `0` confirms the response was served from cache.

Key Concept – Service Discovery: Docker Compose places all services on a shared network by default. Services can reach each other using their service name as a hostname (e.g., `nginx`). Only services with `ports` mappings are accessible from the host machine.

Cleanup:

```
docker compose down
```

Exercise 5: Deploying a Real-World Application

Goal: Learn to read official documentation and deploy a complex, self-hosted service of your choice using Docker Compose.

1. From your working directory, create the File Structure:

```
mkdir ex05
cd ex05
```

2. Choose a Service, go to LinuxServer.io and browse their list of popular images. Choose one that interests you, for example:

- **Jellyfin:** A media server for your movies and music.
- **Nextcloud:** A personal cloud for files, contacts, and calendars.
- **Home Assistant:** An open-source home automation platform.

3. Read the Documentation, on the page for your chosen image, find the "Docker Compose" section. Read it carefully, paying close attention to:

- **Volumes (`- ./config:/config`):** This is where the application's configuration and data will be stored on your host machine. This ensures your data persists even if the container is removed.
- **Environment Variables (`PUID`, `PGID`, `TZ`):** These are critical for correct operation.
  - `TZ` sets your timezone (e.g., `Europe/Lisbon`). You can find your timezone string at Wikipedia: List of tz database time zones.
  - `PUID` and `PGID` ensure that files created by the container have the correct ownership on the host. Find your values by running `id` in your terminal. A common value is `1000`.
- **Ports:** Note which port the application listens on so you know where to access it in your browser.

4. Create Your `compose.yml`, based on the documentation, create the file. Here is an example for **Jellyfin**:

```yaml
services:
  jellyfin:
    image: lscr.io/linuxserver/jellyfin:latest
    container_name: jellyfin
    environment:
      - PUID=1000
      - PGID=1000
      - TZ=Europe/Lisbon
    volumes:
      - ./config:/config
      - ./tvshows:/data/tvshows
      - ./movies:/data/movies
    ports:
```

```
      - "8096:8096"
    restart: unless-stopped
```

5. Prepare and Deploy:
   - Create the local folders you defined in your volumes:

     ```
     mkdir -p config tvshows movies
     ```

   - Start the application:

     ```
     docker compose up -d
     ```

   - Monitor the startup logs to check for errors:

     ```
     docker compose logs -f
     ```

     Press `Ctrl+C` to stop following the logs (the container keeps running).

6. Check the documentation for the default port number. For Jellyfin, it is `8096`. Open your browser to `http://localhost:8096` and follow the setup wizard for your new service.

Verification:

- Run `docker compose ps` to confirm the container is running.
- Run `docker compose logs` to check for any error messages during startup.
- If the web interface does not load, wait a minute – some applications take time to initialize on first launch.

Tip: The `restart: unless-stopped` policy means the container will automatically restart if it crashes or if the Docker daemon restarts (e.g., after a reboot), unless you explicitly stop it with `docker compose down` or `docker compose stop`.

Cleanup:

```
docker compose down
```

Note: this only stops and removes the containers. Your data in the volume folders (`config`, `tvshows`, `movies`) is preserved on the host and will be reused if you start the service again.

Quick Reference: Essential Docker Compose Commands

The following table summarizes the most useful Docker Compose commands you will need throughout these exercises and beyond.

Command	Description
`docker compose up -d`	Start all services in detached (background) mode
`docker compose up --build -d`	Rebuild images and start all services
`docker compose down`	Stop and remove all containers and networks
`docker compose down -v`	Same as above, but also remove named volumes
`docker compose ps`	List running services and their status
`docker compose logs`	Show logs from all services
`docker compose logs -f <service>`	Follow (tail) logs for a specific service
`docker compose exec <service> sh`	Open a shell inside a running container
`docker compose stop`	Stop services without removing containers
`docker compose start`	Start previously stopped services
`docker compose restart`	Restart all services
`docker compose pull`	Pull the latest images for all services
`docker compose config`	Validate and display the resolved compose file

Troubleshooting Tips

If you run into problems during the exercises, try these steps:

1. **Port already in use:** If you see an error like “port is already allocated”, another service (or a previous exercise) is using that port. Stop it first with `docker compose down` in the other exercise folder, or choose a different host port (e.g., change `"8080:80"` to `"8081:80"`).
2. **Container exits immediately:** Check the logs to understand why:

```
docker compose logs <service-name>
```

3. **Changes to files not reflected:** If you modified a `Dockerfile` or files copied with `COPY`, you must rebuild:

   ```
   docker compose up --build -d
   ```

   If you are using volumes (bind mounts), changes should appear immediately – try a hard refresh in your browser (`Ctrl+Shift+R`).

4. **Cannot connect to Docker daemon:** Make sure the Docker service is running:

   ```
   sudo systemctl start docker
   sudo systemctl status docker
   ```

5. **Disk space issues:** Docker images and containers can accumulate over time. Clean up unused resources with:

   ```
   docker system prune
   ```

   Add the `-a` flag to also remove unused images (not just dangling ones).

6. **Inspecting a container:** To explore what is happening inside a running container, open a shell:

   ```
   docker compose exec <service-name> sh
   ```

   This is useful for checking file contents, testing network connectivity (`ping`, `wget`), or reading logs inside the container.